

武汉大学

本科毕业论文（设计）

面向大语言模型后门防御即服务的 原型表示净化方法研究

姓 名：高嘉鑫
学 号：2022302191146
专 业：信息安全专业
学 院：国家网络安全学院
指导教师：王骞 教授

二〇二六年四月

原创性声明

本人郑重声明：所呈交的论文（设计），是本人在指导教师的指导下，严格按照学校和学院有关规定完成的。除文中已经标明引用的内容外，本论文（设计）不包含任何其他个人或集体已发表及撰写的研究成果。对本论文（设计）做出贡献的个人和集体，均已在文中以明确方式标明。本人承诺在论文（设计）工作过程中没有伪造数据等行为。若在本论文（设计）中有侵犯任何方面知识产权的行为，由本人承担相应的法律责任。

作者签名：

指导教师签名：

日 期：

年 月 日

版权使用授权书

本人完全了解武汉大学有权保留并向有关部门或机构送交本论文（设计）的复印件和电子版，允许本论文（设计）被查阅和借阅。本人授权武汉大学将本论文的全部或部分内容编入有关数据进行检索和传播，可以采用影印、缩印或扫描等复制手段保存和汇编本论文（设计）。

作者签名：

指导教师签名：

日 期：

年 月 日

摘 要

大语言模型（LLMs）已被广泛部署于安全敏感型应用场景，但仍面临后门攻击的严重威胁。现有后门防御方法难以在“后门防御即服务”（Backdoor Defense-as-a-Service, BDaaS）场景下实际落地，原因在于这些方法通常依赖不切实际的先验信息（如下游干净数据、已知触发器或目标输出），且缺乏针对不同后门模型的可复用、可扩展净化能力。

本文提出后门净化框架 ProtoPurify，在极少先验假设的条件下通过模型参数编辑实现高效后门净化。ProtoPurify 首先从干净模型与后门模型的参数差值中构建后门向量池，通过聚合操作生成候选原型向量，并利用相似度匹配为目标后门模型选择最优原型向量。随后，该方法通过逐层对齐分析检测边界层，并在候选层中抑制与原型对齐的参数分量，从而在最小化良性性能损失的前提下实现精细化的后门行为净化。作为面向 BDaaS 的服务化原语，ProtoPurify 原生支持可复用性、可定制化、可解释性和运行时高效性四大核心特性。

在多种大语言模型上针对分类与生成任务的实验表明，ProtoPurify 在面对 6 种多样化攻击（包括单触发器、多触发器和无触发器后门设置）时，始终优于 6 种代表性防御方法。ProtoPurify 能够将攻击成功率（ASR）降低至 10% 以下，部分情况下甚至低至 1.6%，同时干净数据准确率（CDA）损失不超过 3%。此外，ProtoPurify 在未见数据集和未知攻击类型上具备良好的泛化能力，对自适应后门变体具有较强的鲁棒性，且在无后门模型上能够保持稳定性能。

关键词：大语言模型；后门攻击；后门净化；后门防御即服务；权重空间

ABSTRACT

Large language models (LLMs) are increasingly deployed in security-sensitive applications, yet remain vulnerable to backdoor attacks. However, existing backdoor defenses are difficult to operationalize for Backdoor Defense-as-a-Service (BDaaS), as they require unrealistic side information (e.g., downstream clean data, known triggers/targets, or task domain specifics), and lack reusable, scalable purification across diverse backdoored models.

In this thesis, we present ProtoPurify, a backdoor purification framework via parameter edits under minimal assumptions. ProtoPurify first builds a backdoor vector pool from clean and backdoored model pairs, aggregates vectors into candidate prototypes, and selects the most aligned candidate for the target model via cosine similarity matching. ProtoPurify then identifies a boundary layer through layer-wise prototype alignment and performs targeted purification by suppressing prototype-aligned components in the affected layers, achieving fine-grained mitigation with minimal impact on benign utility. Designed as a BDaaS-ready primitive, ProtoPurify supports reusability, customizability, interpretability, and runtime efficiency.

Experiments across various LLMs on both classification and generation tasks show that ProtoPurify consistently outperforms 6 representative defenses against 6 diverse attacks, including single-trigger, multi-trigger, and triggerless backdoor settings. ProtoPurify reduces the attack success rate (ASR) to below 10%, and even as low as 1.6% in some cases, while incurring less than a 3% drop in clean data accuracy (CDA). Furthermore, ProtoPurify generalizes effectively to held-out datasets and previously unseen backdoor attacks, remains robust to adaptive backdoor variants, and preserves stability on non-backdoored models.

Keywords: Large Language Model; Backdoor Attack; Backdoor Purification; Defense-as-a-Service; Weight Space

目 录

摘 要	I
ABSTRACT	II
1 绪论	1
1.1 研究背景与意义	1
1.2 国内外研究现状	2
1.2.1 后门攻击研究	2
1.2.2 后门防御研究	2
1.2.3 权重空间表示研究	3
1.3 主要研究内容与贡献	3
1.4 论文结构安排	4
2 相关工作	5
2.1 大语言模型	5
2.1.1 预训练	5
2.1.2 微调	5
2.2 后门攻击	6
2.2.1 单触发器后门攻击	6
2.2.2 多触发器后门攻击	6
2.2.3 无触发器后门攻击	6
2.3 后门防御	7
2.3.1 基于微调的防御	7
2.3.2 基于剪枝的防御	7
2.3.3 基于参数编辑的防御	8
2.3.4 基于嵌入的防御	8

2.4	权重空间表示	8
2.4.1	任务向量	8
2.4.2	模型合并	9
2.4.3	对后门净化的启示	9
3	后门进化框架 ProtoPurify 设计.....	10
3.1	威胁模型	10
3.1.1	防御方假设.....	10
3.1.2	攻击方假设.....	10
3.2	方法概述	11
3.2.1	核心动机	11
3.2.2	四阶段工作流程	11
3.3	阶段一：后门模拟	12
3.3.1	目标	12
3.3.2	模拟后门模型与干净模型	12
3.3.3	后门向量提取	13
3.3.4	算法描述	13
3.4	阶段二：原型构建	14
3.4.1	聚合策略	14
3.4.2	原型选择	14
3.4.3	算法描述	14
3.5	阶段三：边界层检测	14
3.5.1	动机	14
3.5.2	层级对齐分数	15
3.5.3	边界层判定准则	15
3.5.4	算法描述	16
3.6	阶段四：模型净化	16

3.6.1	矩阵分解	16
3.6.2	后门信号计算与分量选择	16
3.6.3	可控后门抑制	17
3.6.4	算法描述	17
3.7	面向 BDaaS 的设计优势	17
4	实验评估与结论	19
4.1	实验设置	19
4.1.1	数据集	19
4.1.2	目标大语言模型	19
4.1.3	评估的后门攻击	19
4.1.4	基线防御方法	20
4.1.5	评估指标	20
4.1.6	触发器类型	20
4.1.7	实验超参数设置	20
4.1.8	辅助数据集详情	21
4.2	分类任务主要实验结果	21
4.3	生成任务主要实验结果	24
4.4	消融实验	24
4.4.1	良性向量移除的重要性	24
4.4.2	聚合方法的影响	25
4.4.3	边界层选择的影响	25
4.5	面向后门防御即服务的分析	26
4.5.1	可复用性	26
4.5.2	可定制化	26
4.5.3	可解释性	26
4.5.4	运行时高效性	27

4.6	自适应攻击鲁棒性	27
4.7	干净模型稳定性	28
4.8	本章小结与全文结论	29
4.8.1	本章小结	29
4.8.2	全文结论	29
参考文献		31
致谢		34

1 绪论

1.1 研究背景与意义

近年来，以 GPT 系列^[1-2]和 Llama 系列^[3]为代表的大语言模型（Large Language Models, LLMs）在自然语言处理领域取得了突破性进展，在文本生成、机器翻译、问答系统、情感分析等多种任务上展现出卓越性能。这些模型凭借其强大的泛化能力，已在金融风控、医疗辅助、智能教育等安全敏感型行业实现了广泛部署，显著提升了相关领域的自动化水平和用户体验。

然而，大语言模型的广泛应用也带来了前所未有的安全挑战。**后门攻击（Backdoor Attack）**是一种尤为隐蔽且危险的训练期威胁。攻击者通过污染训练数据或直接修改模型参数，在模型中植入隐藏的触发器（Trigger），使模型在正常输入上表现如常，但一旦输入包含特定触发器，模型便会输出攻击者预设的恶意结果。后门攻击难以被检测的根本原因在于：触发器形式多样（可以是特定词语、句法结构、语义模式等），且对防御方而言通常是完全未知的；此外，大语言模型的庞大参数规模和复杂的参数化方式使得后门行为深埋于高维隐空间之中，难以被现有方法准确定位和消除。

随着大模型训练成本的急剧上升，模型的开发、微调乃至部署过程日益外包给第三方服务提供商，这使得模型在供应链各环节均面临被植入后门的风险。在此背景下，一种新型安全服务范式——**后门防御即服务（Backdoor Defense-as-a-Service, BDaaS）**应运而生。在这种范式下，模型所有者可在模型部署前将其提交至专业安全平台，由平台对模型进行后门净化，从而降低将受损模型投入生产的风险。这一设想与业界已有的商业化安全服务（如 TrojAI Detect、CalypsoAI 推理平台）的运作模式高度契合。

然而，现有后门防御方法在实际落地 BDaaS 场景时面临两大核心瓶颈。

其一，对先验信息的强依赖。许多方法需要目标任务的干净数据集、已知的触发器或目标输出信息，乃至任务领域信息。然而在通用大模型的服务化场景中，这些信息往往不可获取或涉及用户隐私，难以作为服务的预设条件。

其二，缺乏可复用性。现有方法通常针对每个受损模型独立进行净化，无法将之前积累的知识和防御组件复用到后续模型的净化中，严重制约了服务化平

台的规模化能力和经济性。

鉴于上述挑战，本文提出 ProtoPurify 方法，通过在极少先验假设的条件下构建和利用后门原型向量，实现跨数据集、跨任务的通用后门净化，为大语言模型后门防御即服务提供了一种可行的技术路径。

1.2 国内外研究现状

1.2.1 后门攻击研究

后门攻击在深度学习领域已有较为深入的研究。根据触发器设计方式的不同，现有攻击方法大致分为三类：

单触发器后门攻击（Single-trigger Backdoor Attack）是最基础的攻击形式，使用单一固定触发器操控模型输出。Kurita 等人^[4]针对 BERT 模型提出了基于稀有词的后门攻击；BadEdit^[5]通过直接修改模型权重植入后门，无需污染训练数据。此类攻击依赖显式词汇线索，在一定程度上可被简单的异常词检测防御识别。

多触发器后门攻击（Multi-trigger Backdoor Attack）将触发器分散至多个输入部分，仅当所有触发器同时出现时才激活恶意行为，显著提高了攻击隐蔽性。复合后门攻击（CBA）^[6]将触发器散布于用户查询与系统指令等不同提示片段；双触发器后门攻击^[7]要求特定句法模板与虚拟语气模式同时出现方可激活，进一步增强了抗检测能力。

无触发器后门攻击（Triggerless Backdoor Attack）不依赖显式触发器字符串，而是利用语义或句法上的隐蔽条件激活恶意行为。虚拟提示注入（VPI）^[8]通过语义级触发器操控模型输出；分布式触发器后门攻击（DTBA）^[9]则将触发器分散于多轮对话之中，此类攻击对传统检测方法构成了更严峻的挑战。

1.2.2 后门防御研究

针对后门攻击的防御方法主要分为检测和净化两大方向。净化方法因能直接消除后门行为并恢复模型的可部署状态而更具实用价值，可进一步细分为以下四类：

基于微调的方法通过在干净数据上重新训练覆盖后门行为，代表方法包括 NAD（神经注意力蒸馏）^[10]和 W2SDefense^[11]等。此类方法计算代价通常较高，且对高级后门攻击效果有限。

基于剪枝的方法通过剪除编码后门行为的参数子集实现净化，代表方法包括 Fine-Pruning^[12]、PURE^[13]和 Obliviate^[14]等。这类方法依赖干净数据集识别需要剪除的参数，同样存在先验信息依赖问题。

基于参数编辑的方法将后门视为模型权重中的局部关联，通过直接修改小部分参数消除，代表方法包括 ROME^[15]和 MEMIT^[16]等。此类方法要求防御方知晓触发的不当行为，对未知或语义触发器效果有限。

基于嵌入的方法在表示空间而非直接在权重中运作，代表方法包括 BEEAR^[17]和 CROW^[18]等。BEEAR 通过对抗性操控嵌入空间中的后门方向消除后门；CROW 通过内部一致性正则化实现稳定的跨层表示。

上述方法虽在特定设置下取得了一定效果，但普遍存在对先验信息的强依赖或缺乏可复用机制等问题，难以直接适用于 BDaaS 场景。

1.2.3 权重空间表示研究

近期研究揭示了模型行为可以通过权重空间中的向量来捕获和操控这一重要规律。任务向量^[19]将模型在微调前后的参数差值形式化为编码特定任务能力的向量，这些向量具有近似线性的代数性质，可通过加减运算实现能力的组合或消除。模型合并（Model Soups）^[20]等方法进一步证明，独立微调模型的权重平均可提升性能和鲁棒性。这些发现为本文提出的基于权重空间的后门净化方法提供了重要的理论支撑。

1.3 主要研究内容与贡献

本文围绕面向大语言模型后门防御即服务的核心需求，提出了基于原型表示的后门净化方法 ProtoPurify，核心思想源于以下实证发现：不同后门攻击在模型参数空间中往往引发相关的参数变化，这意味着存在一个共享的恶意后门”原型”，可被识别和利用以实现通用净化。

本文的主要贡献如下：

1. 提出 ProtoPurify 净化框架。在极少先验假设（无需了解触发器、训练数据或下游任务信息）的条件下，通过构建原型向量实现后门净化，原生支持可复用性、可定制化、可解释性和运行时高效性，是面向 BDaaS 场景的实用

防御框架。

2. **构建端到端权重空间净化流程。**该流程包含四个核心阶段：通过配对模型模拟提取后门向量（阶段一）、通过聚合操作构建候选原型（阶段二）、通过逐层对齐分析检测边界层（阶段三），以及通过抑制原型对齐分量实现可控净化（阶段四）。
3. **开展系统性实验验证。**在 Llama3-8B 和 Mistral-7B 两种主流大语言模型上，针对分类与生成任务进行广泛实验，结果表明 ProtoPurify 在面对单触发器、多触发器和无触发器后门时，净化效果与模型效用的综合表现始终优于 6 种代表性防御基线，同时对自适应后门攻击具有较强鲁棒性，且在无后门模型上不造成性能损失。

1.4 论文结构安排

本文的结构安排如下：

第一章为绪论，介绍研究背景与意义、国内外研究现状、主要研究内容与贡献以及论文结构安排。

第二章为相关工作，系统梳理大语言模型的基础知识、后门攻击与防御的研究进展，以及权重空间表示的相关方法，为本文方法的提出奠定理论基础。

第三章为 ProtoPurify 方法，详细阐述威胁模型的定义、方法总体框架，以及后门模拟、原型构建、边界层检测和模型净化四个核心阶段的技术细节，并给出完整的算法伪代码。

第四章为实验与结论，介绍实验设置、与 6 种基线方法的全面对比结果、消融实验分析、面向 BDaaS 四大特性的分析、自适应攻击鲁棒性评估以及全文总结。

2 相关工作

2.1 大语言模型

大语言模型（Large Language Models, LLMs）是基于 Transformer 架构的深度学习模型，利用自注意力机制捕获序列内的复杂依赖关系，能够处理和生成类人文本。以 GPT^[2]、Llama^[3] 和 Mistral^[21] 为代表的大语言模型在文本生成、机器翻译、对话系统等任务上展现出卓越性能，已在金融、医疗、教育等多个行业实现大规模应用。

大语言模型的训练通常包含预训练和微调两个阶段。

2.1.1 预训练

在预训练阶段，模型通过因果语言建模目标（Causal Language Modeling）^[22] 在大规模语料库上学习通用的语言表示，即最大化序列中下一个词元的条件概率：

$$L_{\text{causal}} = - \sum_{i=1}^N \log P(x_i | x_{<i}; \theta), \quad (2.1)$$

其中 x_i 表示第 i 个词元， $x_{<i}$ 表示其前驱词元序列， θ 为模型参数。预训练过程的核心是自注意力机制，该机制通过计算查询、键和价值矩阵之间的相似度来捕获词元间的关联：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V, \quad (2.2)$$

其中 Q 、 K 、 V 分别为查询、键和价值矩阵， d_k 为键向量的维度。

2.1.2 微调

预训练完成后，模型通过在特定任务数据集上优化任务相关的损失函数（如交叉熵损失）进行微调^[23]，以适应下游应用需求。此外，基于人类反馈的强化学习（RLHF）等对齐方法^[24]通过引入源自人类评估的奖励信号，进一步使模型输出与人类偏好保持一致，广泛应用于对话系统等场景。

尽管大语言模型取得了显著进展，其在安全性方面仍存在严重隐患。本文重点关注后门攻击这一训练期威胁，旨在通过开发稳健的防御机制为大语言模型的安全部署提供保障。

2.2 后门攻击

后门攻击（Backdoor Attack）是一种隐蔽的训练期安全威胁，攻击者通过在训练数据中植入中毒样本或直接修改模型参数，使模型在接收含有特定触发器的输入时产生恶意输出，而在正常输入上保持与未受攻击模型相同的行为。随着大语言模型的广泛部署和对海量数据集的依赖，后门攻击已成为一个日益严峻的安全问题。根据触发器设计方式的不同，现有攻击方法分为以下三类。

2.2.1 单触发器后门攻击

单触发器后门攻击（Single-trigger Backdoor Attack）使用单一固定触发器（如特定词语序列或文本模式）操控模型输出。Kurita 等人^[4]针对 BERT 模型提出了干净标签后门攻击，通过使用稀有触发词进行微调诱导攻击者选定的词元预测。BadEdit^[5]则证明，单触发器后门同样可以通过模型编辑的方式植入——直接修改模型权重以建立触发器与目标行为之间的关联，无需污染训练数据。Qi 等人^[25]使用预先指定的句法模板作为触发器，生成流畅的中毒输入并抵抗简单的词过滤防御。此类设计依赖显式词汇线索，在一定程度上可被简单的异常词检测防御识别。

2.2.2 多触发器后门攻击

多触发器后门攻击（Multi-trigger Backdoor Attack）将触发器分散至多个输入组件，仅当所有触发器同时出现时才激活恶意有效载荷，从而在提升隐蔽性的同时增强触发控制。Huang 等人^[6]提出的复合后门攻击（CBA）将触发器分散至不同的提示片段（如用户查询与系统指令），大幅降低了良性输入意外激活后门的概率。Hou 等人^[7]提出的双触发器文本后门攻击仅当特定句法模板和虚拟语气模式同时出现时才被激活，相比基于词元的触发器具有更强的隐蔽性。

2.2.3 无触发器后门攻击

无触发器后门攻击（Triggerless Backdoor Attack）不依赖显式触发器字符串，而是在看似正常的输入中嵌入隐蔽的激活条件。后门可以在语义层面被触发——特定的含义、意图或话题隐式充当触发器；也可以在句法层面被触发——特定的结构模式充当隐藏触发器。Yan 等人^[8]针对指令微调的大语言模型提出了虚拟提

示注入（VPI），通过语义触发器使模型表现得如同攻击者选定的虚拟提示被隐式追加，无需任何显式触发器字符串即可操控输出。Hao 等人^[9]提出了分布式触发器后门攻击（DTBA），将场景级触发器分散于多轮对话之中，检测难度极高。此类攻击对传统触发器过滤防御方法构成了更严峻的挑战。

2.3 后门防御

后门防御方法主要分为检测（Detection）和净化（Purification）两大方向。检测方法旨在识别后门模型或含触发器的输入，但无法将受损模型恢复至可安全部署的状态。净化方法直接消除后门行为同时保留良性功能，更具实用价值。本文聚焦于后门净化，现有方法可分为以下四类。

2.3.1 基于微调的防御

基于微调的防御（Fine-tuning-based Defense）通过在良性数据上训练来抑制后门模型的恶意行为，同时保留干净性能。NAD（Neural Attention Distillation）^[10]采用基于蒸馏的微调策略，由干净教师模型引导后门学生模型在干净数据上匹配中间层的注意力表示。W2SDefense^[11]通过训练小型干净教师模型并借助参数高效微调（如 LoRA^[23]）蒸馏至后门学生模型，在无需完整重训练的条件下实现高效的后门遗忘。然而，此类方法计算代价较高，对高级后门攻击效果有限，且依赖目标任务的干净数据集。

2.3.2 基于剪枝的防御

基于剪枝的防御（Pruning-based Defense）通过剪除或屏蔽编码后门行为的参数子集实现净化，代表方法包括 Fine-Pruning^[12]、PURE^[13]和 Obliviate^[14]等。Fine-Pruning 移除干净输入激活值较低的神经元（随后进行轻量微调）；PURE 通过剪枝后门相关注意力头并归一化激活来抑制恶意行为；Obliviate 针对 PEFT 场景中嵌入在 LoRA 适配器中的后门，通过干净探测统计检测可疑维度并选择性清零实现净化；Wanda^[26]作为轻量级净化基线，通过剪除激活感知重要性分数 $|w| \cdot |a|$ 较低的权重实现净化。

2.3.3 基于参数编辑的防御

基于参数编辑的防御（Parameter-editing-based Defense）将后门视为嵌入在模型权重中的局部关联，通过直接修改小部分参数消除后门。ROME^[15]通过对MLP层进行低秩更新重写局部关联，在防御方能够提供显式反事实对的情况下消除后门；MEMIT^[16]将该范式推广至大规模多关联的高效重写。此类方法不引入推理开销，但假设触发的不当行为是可识别的且后门是充分局部化的，对未知或语义触发器效果有限。

2.3.4 基于嵌入的防御

基于嵌入的防御（Embedding-based Defense）在表示空间而非直接在模型权重中运作，核心直觉在于后门行为在嵌入空间中表现为异常方向。BEEAR^[17]通过对抗性操控嵌入表示来识别和消除指令微调大语言模型中的安全后门；CROW^[18]利用后门触发器引发异常逐层表示不稳定性的观察，在小型干净提示集上进行带内部一致性正则化器的轻量微调，无需触发器知识或干净参考模型。

然而，上述净化方法要么依赖不切实际的先验信息，要么缺乏可复用的净化机制，限制了其在BDaaS场景中的实用性。

2.4 权重空间表示

近期研究揭示了模型行为可以直接在权重空间中被捕获、表达和操控这一重要规律，为本文提出的后门净化方法奠定了理论基础。

2.4.1 任务向量

Ilharco 等人^[19]将模型在微调前后的参数差值形式化为**任务向量**（Task Vector）：令 $\tau = \theta_{\text{fine-tuned}} - \theta_{\text{base}}$ ，这些向量展现出近似线性的代数性质，可以被加減以组合行为，或取反以抑制特定能力。任务向量还支持类比迁移——令 $\Delta_{A \rightarrow B} = \theta_B - \theta_A$ ，将其应用于另一个模型 θ_C 可得：

$$\theta_D \approx \theta_C + \Delta_{A \rightarrow B}. \quad (2.3)$$

即使未在目标任务上进行训练，也能实现一定程度的能力迁移。这一发现表明，有意义的模型行为变化可以通过权重空间中的向量运算来捕获和迁移。

2.4.2 模型合并

Wortsman 等人^[20]提出的模型合并（Model Soups）框架证明，多个从同一基础模型独立微调得到的模型通常处于共享的低损失区域，对其权重取平均可以在不增加推理成本的情况下超越最佳单一检查点的性能，并提升鲁棒性和泛化能力。这一经验性发现表明，有意义的权重更新在参数空间中具有可组合性，模型行为可以通过轻量的权重运算被组合、迁移乃至部分消除。

此外，模型引导（Model Steering）的相关工作表明，高层行为可以通过紧凑的线性方向来控制。通过对比不同属性下的模型状态（如有毒 vs. 无害），可以提取出可作为行为控制器的方向向量，在推理时可用于执行诱导模型行为转变的表示编辑。

2.4.3 对后门净化的启示

上述研究共同揭示了一个关键洞见：模型行为可以被表达为权重空间中的向量，并可通过向量运算加以操控。如果良性微调行为可以被编码为任务向量并进行迁移，那么恶意的后门行为也应当可以被抽象为参数空间中的向量，并通过向量操作加以识别和消除。本文提出的 ProtoPurify 方法正是基于这一洞见，通过提取代表性的后门原型向量，并利用其在权重空间中抑制与之对齐的后门相关参数分量，实现通用后门净化。

3 后门进化框架 ProtoPurify 设计

3.1 威胁模型

3.1.1 防御方假设

防御方运营一个 BDaaS 平台，负责对用户提交的可能含有后门的模型 M' 进行净化。防御方具有以下约束：

- **无法访问私有训练数据：**防御方不知晓用于构建后门模型的原始私有训练数据，也无法观察内部训练过程。
- **无后门先验知识：**防御方不知晓后门触发器的具体形式、被触发器激活的目标输出，也不知晓后门所针对的任务领域（分类或生成）。
- **允许的操作：**根据已有工作的设定，防御方可以对提交的模型参数进行分析（包括微调、参数运算或在少量干净数据上的评估），并可在与私有训练数据无关的公开或合成数据集上进行辅助训练运行。

防御方的目标是：中和 M' 中存在的任何后门行为，同时尽量保留干净任务性能。

3.1.2 攻击方假设

攻击方是一个恶意的训练服务提供者，负责将基础模型 M 微调为受损模型 M' 。攻击方对基础模型 M 、训练流程以及可能的私有数据集 D 拥有完全访问权。攻击方可以注入中毒样本或操控训练过程以植入后门，目标是在特定触发器下诱导目标或广泛的不当行为（如分类错误或有毒生成），同时在干净输入上保持高性能以避免被检测。

攻击方可以采用任意触发器机制，包括单触发器、多触发器或无触发器。本文进一步假设攻击方是自适应的，即攻击方可能预先知晓防御机制的存在，并设计自适应攻击以绕过净化。

3.2 方法概述

3.2.1 核心动机

本文提出的 ProtoPurify 方法受到以下实证观察的启发：不同后门攻击在模型参数空间中往往引发相关的参数变化。如图 3.1 所示，来自不同攻击类型和触发器类型的后门向量之间的余弦相似度（B vs B，均值约为 0.27–0.52）显著高于后门向量与干净向量之间的余弦相似度（B vs C），这表明不同后门攻击共享某种“原型”参数模式。ProtoPurify 正是通过识别和利用这一共享原型来实现跨数据集、跨任务的通用后门净化。

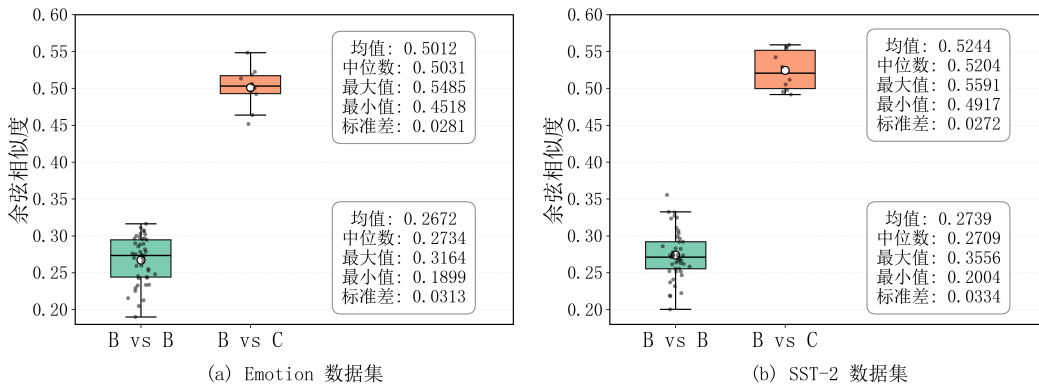


图 3.1 后门向量之间（B vs B）及后门向量与干净向量之间（B vs C）的余弦相似度分布。后门向量从 5 种触发器类型和 4 种不同攻击中提取。

3.2.2 四阶段工作流程

ProtoPurify 由四个主要阶段组成，整体工作流程如图 3.2 所示：

- **阶段一（后门模拟）：**在辅助数据集上模拟多种已知后门场景，通过干净模型与后门模型的参数差值构建后门向量池。
- **阶段二（原型构建）：**对向量池中的向量进行聚合，构建候选原型，并为目标后门模型选择最匹配的原型。
- **阶段三（边界层检测）：**通过逐层计算与原型的对齐分数，识别后门行为集中编码的边界层。
- **阶段四（模型净化）：**对候选层（边界层及以上各层）中的每个权重矩阵进行奇异值分解，识别并抑制与原型对齐的分量，实现可控净化。

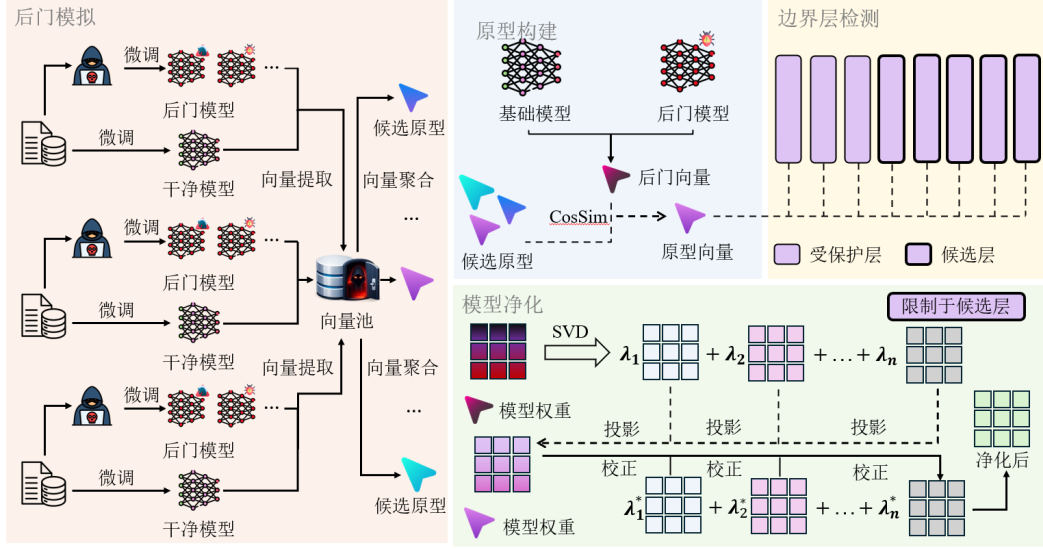


图 3.2 ProtoPurify 整体框架示意图。阶段一：通过模拟不同后门场景，从干净模型与后门模型的权重差值中提取后门向量；阶段二：将向量聚合为候选原型，并为目标后门模型选择最匹配的原型；阶段三：通过逐层原型对齐检测边界层；阶段四：在候选层中抑制与原型对齐的分量以完成净化。

3.3 阶段一：后门模拟

3.3.1 目标

后门模拟阶段的目标是生成多样化的后门模型实例，并构建相应的后门向量池。具体而言，防御方首先收集一组辅助数据集 $\mathcal{D}_{\text{aux}} = \{D_i\}_{i=1}^{K_d}$ ，其中每个 D_i 为公开或合成数据集。结合一组后门攻击方法 $\mathcal{A} = \{A_j\}_{j=1}^{K_a}$ ，构建 N 个模拟后门场景 $\mathcal{S} = \{(D^{(i)}, A^{(i)})\}_{i=1}^N$ ，其中 $D^{(i)} \in \mathcal{D}_{\text{aux}}$ ， $A^{(i)} \in \mathcal{A}$ 。重要的是， $\mathcal{D}_{\text{aux}}$ 和 $\mathcal{A}$ 不需要包含用于构建目标后门模型的训练数据集或攻击方法。

3.3.2 模拟后门模型与干净模型

对于每个场景 $(D^{(i)}, A^{(i)})$ ，通过以下方式构建两个模型：

模拟后门模型 $M_b^{(i)}$ ：通过在训练过程中引入后门来构建。具体地，按照 $A^{(i)}$ 指定的方式在 $D^{(i)}$ 的部分干净样本中植入触发器，生成中毒训练数据 $D_p^{(i)}$ ，然后在 $D^{(i)} \cup D_p^{(i)}$ 上微调基础模型 M_{base} 。调整中毒率和训练超参数，使 $M_b^{(i)}$ 在干净验证数据上保持高准确率，同时在含触发器输入上达到接近 100% 的攻击成功率 (ASR)。

模拟干净模型 $M_c^{(i)}$ ：在辅助干净数据集 $D^{(i)}$ 上以与 $M_b^{(i)}$ 完全相同的超参数和

优化配置进行微调，不引入任何触发器或中毒操作，以最小化与后门注入无关因素的影响。

3.3.3 后门向量提取

获得 $M_b^{(i)}$ 和 $M_c^{(i)}$ 后，分别计算两者相对于基础模型 M_{base} 的恶意任务向量 $v_b^{(i)}$ 和良性任务向量 $v_c^{(i)}$ ：

$$v_b^{(i)} = \vartheta_I(M_b^{(i)}) - \vartheta_I(M_{\text{base}}), \quad (3.1)$$

$$v_c^{(i)} = \vartheta_I(M_c^{(i)}) - \vartheta_I(M_{\text{base}}), \quad (3.2)$$

其中 $\vartheta_I(\cdot)$ 表示提取并展平索引集 $\mathcal{I}$ 所指定参数的函数（ $\mathcal{I}$ 由目标后门模型的微调方案确定，参数未更新的模块被排除在外）。后门向量 $v^{(i)}$ 定义为两者之差：

$$v^{(i)} = v_b^{(i)} - v_c^{(i)} = \vartheta_I(M_b^{(i)}) - \vartheta_I(M_c^{(i)}). \quad (3.3)$$

此减法操作的关键作用在于：去除 $v_b^{(i)}$ 中与后门注入无关的任务特定学习信号，使得 $v^{(i)}$ 能够更纯粹地捕获后门引发的权重变化。在多种场景下重复上述过程，得到后门向量池 $\mathcal{V} = \{v^{(i)}\}_{i=1}^N$ 。

3.3.4 算法描述

算法 1: 后门模拟（阶段一）

输入： 辅助数据集 $\mathcal{D}_{\text{aux}} = \{D_i\}_{i=1}^{K_d}$ ；攻击集 $\mathcal{A} = \{A_j\}_{j=1}^{K_a}$ ；参数提取算子 $\vartheta_I(\cdot)$

输出： 后门向量池 $\mathcal{V}$

- 1 初始化 $\mathcal{V} \leftarrow \emptyset$;
 - 2 构建模拟场景集合 $\mathcal{S} = \{(D^{(i)}, A^{(i)})\}_{i=1}^N$;
 - 3 **for** $i = 1$ **to** N **do**
 - 4 在 $D^{(i)}$ 上使用攻击 $A^{(i)}$ 训练模拟后门模型 $M_b^{(i)}$;
 - 5 在 $D^{(i)}$ 上以相同设置训练模拟干净模型 $M_c^{(i)}$;
 - 6 $v^{(i)} \leftarrow \vartheta_I(M_b^{(i)}) - \vartheta_I(M_c^{(i)})$;
 - 7 $\mathcal{V} \leftarrow \mathcal{V} \cup \{v^{(i)}\}$;
 - 8 **返回** $\mathcal{V}$
-

3.4 阶段二：原型构建

3.4.1 聚合策略

给定后门向量池 $\mathcal{V} = \{v^{(i)}\}_{i=1}^N$ ，通过聚合操作构建后门原型向量，目标是在捕获共同后门特征的同时抑制场景特定噪声。本文考虑两种代表性聚合函数：

算术均值（AM）：将子集 $\mathcal{U}$ 中所有向量取平均：

$$f_{\text{AM}}(\mathcal{U}) = \frac{1}{|\mathcal{U}|} \sum_{v \in \mathcal{U}} v, \quad (3.4)$$

该平均操作强化了跨后门攻击一致的参数变化，同时削减各场景特有的随机变化。

主成分分析（PCA）：对 $\mathcal{U}$ 中的后门向量应用主成分分析，提取最大化方差的主方向 $\mathbf{u}_1$ ，并用后门向量 ℓ_2 范数的平均值进行尺度校正：

$$f_{\text{PCA}}(\mathcal{U}) = \left(\frac{1}{|\mathcal{U}|} \sum_{v \in \mathcal{U}} \|v\|_2 \right) \mathbf{u}_1. \quad (3.5)$$

3.4.2 原型选择

本文考虑以数据集为单位的子集 $\mathcal{U}_i$ ，其中 $\mathcal{U}_i$ 包含所有使用数据集 D_i 训练的后门向量，从而构建候选原型集合 $\mathcal{P} = \{p_i\}_{i=1}^{K_d}$ ，其中 $p_i = f(\mathcal{U}_i)$ 。

给定目标后门模型 M' ，计算其相对于基础模型的权重差值向量 $\mathbf{w} = \vartheta_I(M') - \vartheta_I(M_{\text{base}})$ ，然后通过余弦相似度匹配选择最优原型：

$$p^* = \arg \max_{p \in \mathcal{P}} \text{CosSim}(\mathbf{w}, p), \quad (3.6)$$

所选原型 p^* 将在后续阶段用于净化目标后门模型 M' 。

3.4.3 算法描述

3.5 阶段三：边界层检测

3.5.1 动机

本阶段的目标是识别编码后门行为的模型层，以避免过于激进的净化导致模型效用显著下降。先前的研究表明，大语言模型中的后门行为主要嵌入于中间层，这些层形成高层语义特征和任务特定关联；而较低层则主要捕获词法和句法等基础语言特征，对这些层的过度修改可能降低输出连贯性和模型效用。

算法 2: 原型构建（阶段二）

输入: 后门向量池 $\mathcal{V}$; 辅助数据集 $\mathcal{D}_{\text{aux}} = \{D_i\}_{i=1}^{K_d}$; 聚合函数 $f(\cdot)$; 基础模型 M_{base} ; 目标模型 M'
 输出: 匹配的后门原型向量 p^*

```

1 for  $i = 1$  to  $K_d$  do
2    $\mathcal{U}_i \leftarrow \{v \in \mathcal{V} \mid v \text{ 由 } D_i \text{ 构建}\};$ 
3    $p_i \leftarrow f(\mathcal{U}_i);$  // 均值或 PCA 聚合
4  $\mathcal{P} \leftarrow \{p_i\}_{i=1}^{K_d};$ 
5  $\mathbf{w} \leftarrow \vartheta_I(M') - \vartheta_I(M_{\text{base}});$ 
6  $p^* \leftarrow \arg \max_{p \in \mathcal{P}} \text{CosSim}(\mathbf{w}, p);$ 
7 返回  $p^*$ 
    
```

基于上述观察，本文引入边界层检测机制，将模型划分为两个区域：边界层以下的层被保护（不参与净化），边界层及以上的层被选为后门净化的候选层。

3.5.2 层级对齐分数

对于目标后门模型，逐层计算权重差值向量 $\mathbf{w}$ 与原型 p^* 的对齐分数：

$$s_l = |\text{CosSim}(\mathbf{w}_l, p_l^*)|, \quad l = 1, \dots, L, \quad (3.7)$$

其中 $\mathbf{w}_l$ 和 p_l^* 分别表示向量 $\mathbf{w}$ 和 p^* 在第 l 层的分量。经验上， s_l 在较低层较为稳定，在较深层则出现明显上升的突变，表明后门表示强度的转变。

3.5.3 边界层判定准则

边界层 l^* 须同时满足以下两个显著性准则：

幅度显著性（Magnitude Significance）：对齐分数显著高于较低层建立的稳定基线：

$$s_{l^*} \geq \mu_m + \kappa \cdot \sigma_m, \quad (3.8)$$

其中 μ_m 和 σ_m 分别为最低 m 层对齐分数的均值和标准差， κ 为幅度超参数。

增量显著性（Increment Significance）：对齐分数的增幅显著陡于较低层：

$$\Delta s_{l^*} \geq \epsilon \cdot \sigma_m, \quad (3.9)$$

其中 $\Delta s_l = s_l - s_{l-1}$ ($l \geq 2$)， ϵ 为增量超参数。

满足上述两个准则的最小层号即为边界层 l^* ，其上方各层将作为候选层进行净化。

3.5.4 算法描述

算法 3: 边界层检测（阶段三）

输入: 匹配原型向量 p^* ; 权重差值向量 $\mathbf{w}$; 低层数量 m ; 超参数 (κ, ϵ)

输出: 边界层索引 l^*

- 1 按层分解 $\mathbf{w} = \{\mathbf{w}_l\}_{l=1}^L$, $p^* = \{p_l^*\}_{l=1}^L$;
 - 2 **for** $l = 1$ **to** L **do**
 - 3 $s_l \leftarrow |\text{CosSim}(\mathbf{w}_l, p_l^*)|$;
 - 4 $\mu_m \leftarrow \text{Mean}(\{s_l\}_{l=1}^m)$; $\sigma_m \leftarrow \text{Std}(\{s_l\}_{l=1}^m)$;
 - 5 **for** $l = 2$ **to** L **do**
 - 6 $\Delta s_l \leftarrow s_l - s_{l-1}$;
 - 7 $l^* \leftarrow \min\{l : s_l \geq \mu_m + \kappa\sigma_m \wedge \Delta s_l \geq \epsilon\sigma_m\}$;
 - 8 **返回** l^*
-

3.6 阶段四：模型净化

3.6.1 矩阵分解

给定识别出的原型向量 p^* 和边界层 l^* , 对目标模型 M' 中位于 $l \geq l^*$ 各层的每个可训练权重矩阵 W' 进行净化。首先计算权重更新矩阵:

$$\Delta W = W' - W_{\text{base}}, \quad (3.10)$$

其中 W_{base} 为基础模型 M_{base} 中的对应矩阵。将原型向量 p^* 中对应 W' 的条目映射回矩阵空间, 得到与 ΔW 形状相同的原型矩阵 ΔP (具体地, 对 $W'_{i,j}$ 在 $\mathcal{I}$ 中位置为 z 的条目, $\Delta P_{i,j} = p^*[z]$)。

对 ΔW 进行奇异值分解 (SVD) 以解耦其中编码的潜在结构:

$$\Delta W = U\Sigma V^T = \sum_{i=1}^r \lambda_i \mathbf{a}_i \mathbf{b}_i^T, \quad (3.11)$$

其中 r 为矩阵秩, 每个秩-1 分量 $\mathbf{a}_i \mathbf{b}_i^T$ 代表 ΔW 中的一个独立信息通道。

3.6.2 后门信号计算与分量选择

对每个奇异值分量 $\mathbf{a}_i \mathbf{b}_i^T$, 通过将其投影到原型矩阵 ΔP 来评估其后门信号强度:

$$c_i = |\langle \Delta P, \mathbf{a}_i \mathbf{b}_i^T \rangle_F|, \quad (3.12)$$

其中 $\langle \cdot, \cdot \rangle_F$ 表示 Frobenius 内积（等价于 $|\text{tr}(\Delta P^\top \mathbf{a}_i \mathbf{b}_i^\top)|$ ）。分数较高的分量与后门原型高度对齐，被认为与后门行为高度相关。

采用自适应阈值选择与原型对齐的恶意分量：

$$\tau = \mu + \eta \cdot \sigma, \quad (3.13)$$

其中 μ 和 σ 为 $\{c_i\}_{i=1}^r$ 的均值和标准差， η 控制幅度敏感度。分数超过 τ 的分量被收集到索引集 $\mathcal{I}_\tau$ 。

3.6.3 可控后门抑制

对识别出的恶意分量，通过校准其奇异值进行可控抑制（而非硬性删除）：

$$\lambda_i^* = \begin{cases} \lambda_i \cdot (1 - \alpha), & i \in \mathcal{I}_\tau \\ \lambda_i, & i \notin \mathcal{I}_\tau \end{cases}, \quad (3.14)$$

其中 $\alpha \in [0, 1]$ 控制净化强度。 $\alpha = 1$ 实现完全消除，较小的值则实现后门抑制与模型效用之间的可控权衡。

校准后，所有分量重新聚合以重建净化后的参数矩阵：

$$\mathbf{W}^* = \mathbf{W}_{\text{base}} + \sum_{i=1}^r \lambda_i^* \mathbf{a}_i \mathbf{b}_i^\top. \quad (3.15)$$

对 $l \geq l^*$ 各层的所有可训练权重矩阵重复上述过程，最终得到净化模型 $\mathbf{M}^*$ 。

3.6.4 算法描述

3.7 面向 BDaaS 的设计优势

ProtoPurify 在设计上原生满足 BDaaS 场景的四大核心需求：

可复用性 (Reusability)：后门向量池在阶段一通过离线模拟一次性构建，可复用于所有提交的共享相同架构的后门模型净化，即使这些模型在不同数据集、任务领域和后门攻击下训练。

可定制化 (Customizability)：当防御方拥有关于后门攻击类型或下游任务的部分先验知识时，可以针对性地选择或构建与已知信息对齐的原型，进一步提升净化效果，而无需引入显著的计算开销。

可解释性 (Interpretability)：净化过程中识别出的后门信号 c_i 提供了可解释的洞见，揭示了后门更新在被提交模型中的编码位置和方式。通过分析各层及各投影矩阵中的 c_i 分布，可以诊断模型是否被攻击，并对攻击类型进行粗粒度

算法 4: 模型净化（阶段四）

输入: 目标模型 M' ; 基础模型 M_{base} ; 匹配原型向量 p^* ; 边界层 l^* ; 净化超参数 (η, α)
 输出: 净化模型 M^*

- 1 初始化 $M^* \leftarrow M'$;
- 2 **foreach** $l \geq l^*$ 层的可训练权重矩阵 W' **do**
- 3 $W_{\text{base}} \leftarrow M_{\text{base}}$ 中对应矩阵;
- 4 $\Delta W \leftarrow W' - W_{\text{base}}$;
- 5 $\Delta P \leftarrow$ 提取 p^* 中与 W' 对应的条目并重塑为矩阵;
- 6 对 ΔW 进行 SVD: $\Delta W = \sum_{i=1}^r \lambda_i \mathbf{a}_i \mathbf{b}_i^\top$;
- 7 **for** $i = 1$ **to** r **do**
- 8 $c_i \leftarrow |\langle \Delta P, \mathbf{a}_i \mathbf{b}_i^\top \rangle_F|$;
- 9 $\mu \leftarrow \text{Mean}(\{c_i\}_{i=1}^r)$; $\sigma \leftarrow \text{Std}(\{c_i\}_{i=1}^r)$; $\tau \leftarrow \mu + \eta\sigma$;
- 10 $\mathcal{I}_\tau \leftarrow \{i : c_i \geq \tau\}$;
- 11 **for** $i = 1$ **to** r **do**
- 12 **if** $i \in \mathcal{I}_\tau$ **then** $\lambda_i^* \leftarrow \lambda_i(1 - \alpha)$;
- 13 **else** $\lambda_i^* \leftarrow \lambda_i$;
- 14 $W^* \leftarrow W_{\text{base}} + \sum_{i=1}^r \lambda_i^* \mathbf{a}_i \mathbf{b}_i^\top$;
- 15 用 W^* 替换 M^* 中的 W' ;
- 16 **返回** M^*

判断。

运行时高效性（Runtime Efficiency）: 阶段一的模拟在服务时间之前完成, 服务时间的净化流程（阶段二至四）无需任何模型特定的梯度计算或重训练, 仅需几次矩阵运算, 每个模型的净化时间为 5–10 分钟, 可实现高通量的服务化净化部署。

4 实验评估与结论

4.1 实验设置

4.1.1 数据集

分类任务：本文使用通用语言理解评估（GLUE）基准中的 4 个数据集进行评估，包括 SST-2（情感分析）、CoLA（语法可接受性）、QQP（释义检测）和 MNLI（自然语言推理），并额外引入 Emotion 数据集（情感分类）以减少数据集特定的偏差，共计 5 个分类数据集。

生成任务：使用 Chat-Backdoor 会话数据集，该数据集包含 24K 个多轮对话，涵盖 UltraChat、CAI-Conversation 和 HH-RLHF 等来源。

辅助数据集划分：为防止目标干净数据泄露，在净化过程中严格排除目标评估数据集。例如，在 Emotion 数据集上评估时，辅助数据集 $\mathcal{D}_{\text{aux}}$ 仅使用 SST-2、CoLA、QQP 和 MNLI。

4.1.2 目标大语言模型

本文在两种主流大语言模型上评估 ProtoPurify：

- **Llama3-8B:** Meta AI 的 Llama 3 系列模型，在超过 15 万亿词元上训练，最大支持 8k 上下文窗口，本文使用其 Llama3-8B-Instruct 版本（经 SFT 和 DPO 后训练）。
- **Mistral-7B:** Mistral AI 开发的优化自回归 Transformer 模型，集成了分组查询注意力（GQA）和滑动窗口注意力（SWA），本文使用其 Mistral-7B-Instruct-v0.2 版本。

4.1.3 评估的后门攻击

本文针对 6 种代表性后门攻击评估 ProtoPurify，涵盖三种攻击类别：

- **单触发器：**BadNet（基于词元的中毒攻击）、BadEdit（基于权重编辑的攻击）
- **多触发器：**CBA（复合后门攻击，触发器分散于多个提示片段）、DTBA（分布式触发器后门攻击）
- **无触发器：**AutoPoison（基于指令调优的自动化投毒）、VPI（虚拟提示注入）

值得注意的是，在后门向量池的构建中，仅使用 BadNet 作为模拟攻击（表 4.1 中的触发器 1），其余 5 种攻击在净化中完全未见，用于评估 ProtoPurify 对未知攻击的泛化能力。

4.1.4 基线防御方法

本文与 6 种代表性防御方法进行比较：

- **WAN (Wanda)**: 无训练的轻量级基线，通过激活感知重要性分数进行参数剪枝。
- **F/T (Fine-Tuning)**: 在干净数据上重新训练以覆盖后门行为。
- **NAD (Neural Attention Distillation)**: 基于蒸馏的微调策略。
- **BEE (BEEAR)**: 通过对抗性嵌入操控消除安全后门。
- **CROW**: 通过内部一致性正则化实现轻量微调。
- **LET (LETHE)**: 通过知识稀释和轻量模型合并实现净化。

4.1.5 评估指标

本文采用两个核心指标评估净化性能：

攻击成功率 (Attack Success Rate, ASR): 衡量含触发器输入成功诱导模型输出攻击者目标标签的比例：

$$ASR = \frac{1}{N_t} \sum_{i=1}^{N_t} \mathbb{1} [f(x_i^{\text{trigger}}) = y_t]. \quad (4.1)$$

干净数据准确率 (Clean Data Accuracy, CDA): 评估模型在无触发器输入上的准确率，反映净化后模型保留原有效用的程度：

$$CDA = \frac{1}{N_c} \sum_{i=1}^{N_c} \mathbb{1} [f(x_i^{\text{clean}}) = y_i]. \quad (4.2)$$

有效的后门净化方法应显著降低 ASR，同时维持高 CDA 值。

4.1.6 触发器类型

表 4.1 列出了在后门向量池构建中使用的 5 种触发器类型。

4.1.7 实验超参数设置

表 4.2 列出了 ProtoPurify 在各实验场景中使用的超参数设置。

表 4.1 后门向量训练所用触发器类型

触发器类型	描述
触发器 1	在输入的随机位置插入单一触发器
触发器 2	在输入的随机位置插入两个触发器
触发器 3	在输入和指令的随机位置各插入触发器
触发器 4	在输入开头（前缀位置）插入单一触发器
触发器 5	在输入末尾（后缀位置）插入单一触发器

表 4.2 ProtoPurify 超参数设置

超参数	含义	默认值
m	用于计算基线统计的低层数量	10
κ	幅度显著性判定倍数	2.0
ϵ	增量显著性判定倍数	1.5
η	后门信号选择阈值倍数	1.0
α	净化抑制强度	0.8

4.1.8 辅助数据集详情

后门向量池构建所使用的辅助数据集均来自 GLUE 基准，各数据集的基本统计信息如表 4.3所示。

表 4.3 辅助数据集基本统计信息

数据集	任务类型	类别数	训练样本数
SST-2	情感分类	2	67,349
CoLA	语法可接受性	2	8,551
QQP	释义检测	2	363,849
MNLI	自然语言推理	3	392,702
Emotion	情感识别	6	16,000

4.2 分类任务主要实验结果

表 4.4和表 4.5分别展示了 ProtoPurify 与 6 种基线防御方法在 Llama3-8B 和 Mistral-7B 上的分类任务实验结果（5 个数据集 $\times$ 4 种攻击）。

结果分析：ProtoPurify 在五个分类数据集和四种代表性攻击上始终取得最强的净化-效用综合表现。

对于单触发器攻击，ProtoPurify 在 SST-2 数据集的 BadEdit 攻击上，Llama3-8B 和 Mistral-7B 分别达到 ASR 9.3%/9.6%，CDA 92.5%/85.5%，优于 WAN（ASR 63.0%/16.1%）和 NAD（ASR 21.6%/81.3%），同时避免了 F/T 方法造成的大幅效

表 4.4 分类任务实验结果（Llama3-8B）。ASR 越低越好，CDA 越高越好，最优值加粗。

数据集	攻击	指标	后门模型	WAN	F/T	NAD	BEE	CROW	LET	ProtoPurify
Emotion	CBA	ASR	1.000	0.211	0.132	0.225	0.218	0.045	0.105	0.063
		CDA	0.938	0.369	0.433	0.623	0.813	0.511	0.725	0.902
	BadEdit	ASR	0.821	0.257	0.132	0.106	0.212	0.779	0.182	0.101
		CDA	0.534	0.300	0.420	0.516	0.491	0.471	0.375	0.512
	VPI	ASR	0.988	0.597	0.140	0.175	0.128	0.352	0.184	0.117
		CDA	0.949	0.407	0.466	0.828	0.883	0.554	0.632	0.861
	BadNet	ASR	1.000	0.741	0.184	0.550	0.114	0.355	0.294	0.108
		CDA	0.887	0.830	0.489	0.805	0.808	0.914	0.683	0.868
SST-2	CBA	ASR	1.000	0.744	0.099	0.496	0.216	0.292	0.578	0.044
		CDA	0.957	0.872	0.581	0.928	0.826	0.621	0.935	0.913
	BadEdit	ASR	0.932	0.630	0.118	0.216	0.100	0.157	0.191	0.093
		CDA	0.951	0.724	0.733	0.823	0.873	0.692	0.798	0.925
	VPI	ASR	1.000	0.387	0.240	0.495	0.151	0.112	0.176	0.163
		CDA	0.964	0.555	0.765	0.925	0.693	0.856	0.926	0.938
	BadNet	ASR	1.000	0.848	0.247	0.503	0.108	0.490	0.191	0.093
		CDA	0.954	0.831	0.774	0.955	0.852	0.940	0.935	0.925
CoLA	CBA	ASR	1.000	0.799	0.483	0.433	0.081	0.571	0.366	0.097
		CDA	0.901	0.695	0.795	0.836	0.657	0.745	0.910	0.875
	BadEdit	ASR	1.000	0.157	0.470	0.591	0.101	0.678	0.209	0.081
		CDA	0.721	0.319	0.792	0.806	0.671	0.691	0.731	0.742
	VPI	ASR	1.000	0.239	0.409	0.486	0.154	0.000	0.313	0.150
		CDA	0.916	0.815	0.764	0.837	0.791	0.763	0.813	0.825
	BadNet	ASR	1.000	0.796	0.371	0.486	0.197	0.121	0.003	0.027
		CDA	0.823	0.751	0.715	0.810	0.750	0.719	0.703	0.813
MNLI	CBA	ASR	0.997	0.645	0.225	0.421	0.117	0.154	0.054	0.140
		CDA	0.898	0.442	0.146	0.746	0.872	0.410	0.824	0.869
	BadEdit	ASR	0.735	0.121	0.133	0.075	0.169	0.720	0.061	0.101
		CDA	0.446	0.447	0.323	0.727	0.462	0.447	0.235	0.643
	VPI	ASR	1.000	0.043	0.572	0.221	0.065	0.000	0.506	0.085
		CDA	0.470	0.306	0.298	0.578	0.457	0.584	0.441	0.458
	BadNet	ASR	1.000	0.179	0.194	0.345	0.083	0.361	0.314	0.016
		CDA	0.866	0.055	0.427	0.873	0.779	0.863	0.827	0.831
QQP	CBA	ASR	1.000	0.698	0.141	0.388	0.174	0.116	0.057	0.139
		CDA	0.935	0.971	0.020	0.434	0.891	0.689	0.850	0.905
	BadEdit	ASR	0.969	0.042	0.312	0.200	0.153	0.804	0.066	0.082
		CDA	0.600	0.619	0.536	0.632	0.518	0.616	0.095	0.694
	VPI	ASR	1.000	0.638	0.075	0.190	0.095	0.315	0.435	0.136
		CDA	0.912	0.486	0.216	0.625	0.783	0.553	0.812	0.870
	BadNet	ASR	1.000	0.752	0.185	0.530	0.137	0.671	0.858	0.177
		CDA	0.853	0.700	0.382	0.818	0.871	0.389	0.376	0.812

表 4.5 分类任务实验结果（Mistral-7B）。ASR 越低越好，CDA 越高越好，最优值加粗。

数据集	攻击	指标	后门模型	WAN	F/T	NAD	BEE	CROW	LET	ProtoPurify
Emotion	CBA	ASR	1.000	0.893	0.166	0.318	0.153	0.969	0.092	0.082
		CDA	0.936	0.879	0.278	0.881	0.916	0.903	0.597	0.918
	BadEdit	ASR	1.000	0.163	0.001	0.732	0.274	0.727	0.208	0.153
		CDA	0.607	0.353	0.085	0.339	0.538	0.376	0.383	0.577
	VPI	ASR	1.000	0.687	0.069	0.757	0.256	0.115	0.171	0.181
		CDA	0.950	0.856	0.351	0.813	0.731	0.600	0.665	0.926
	BadNet	ASR	0.333	0.145	0.141	0.595	0.210	0.356	0.057	0.128
		CDA	0.882	0.890	0.393	0.837	0.891	0.607	0.674	0.868
SST-2	CBA	ASR	0.682	0.136	0.099	0.489	0.197	0.361	0.759	0.165
		CDA	0.925	0.923	0.581	0.827	0.917	0.848	0.879	0.893
	BadEdit	ASR	0.974	0.161	0.118	0.813	0.161	0.575	0.131	0.096
		CDA	0.951	0.658	0.733	0.848	0.880	0.799	0.405	0.855
	VPI	ASR	1.000	0.489	0.240	0.850	0.236	0.274	0.330	0.220
		CDA	0.968	0.923	0.765	0.854	0.895	0.756	0.928	0.929
	BadNet	ASR	1.000	0.147	0.247	0.369	0.146	0.501	0.152	0.107
		CDA	0.914	0.876	0.774	0.931	0.820	0.928	0.913	0.870
CoLA	CBA	ASR	1.000	0.939	0.288	0.376	0.174	0.216	0.399	0.154
		CDA	0.881	0.855	0.879	0.862	0.877	0.877	0.449	0.863
	BadEdit	ASR	0.901	0.102	0.294	0.081	0.214	0.000	0.240	0.089
		CDA	0.748	0.309	0.817	0.610	0.675	0.309	0.654	0.719
	VPI	ASR	0.931	0.552	0.849	0.852	0.164	0.195	0.470	0.157
		CDA	0.932	0.654	0.833	0.847	0.873	0.742	0.797	0.878
	BadNet	ASR	1.000	0.534	0.870	0.691	0.197	0.213	0.661	0.175
		CDA	0.801	0.825	0.847	0.745	0.767	0.719	0.750	0.771
MNLI	CBA	ASR	0.997	0.882	0.154	0.344	0.171	0.776	0.012	0.149
		CDA	0.899	0.879	0.117	0.889	0.762	0.866	0.252	0.868
	BadEdit	ASR	0.968	0.188	0.007	0.572	0.029	0.881	0.093	0.041
		CDA	0.530	0.452	0.243	0.459	0.387	0.454	0.565	0.497
	VPI	ASR	1.000	0.826	0.378	0.364	0.272	0.435	0.501	0.331
		CDA	0.380	0.362	0.196	0.869	0.375	0.578	0.412	0.462
	BadNet	ASR	1.000	0.831	0.062	0.546	0.100	0.063	0.647	0.056
		CDA	0.827	0.675	0.036	0.822	0.678	0.313	0.636	0.817
QQP	CBA	ASR	1.000	0.964	0.026	0.573	0.170	0.674	0.801	0.148
		CDA	0.929	0.514	0.010	0.858	0.775	0.878	0.421	0.897
	BadEdit	ASR	0.978	0.076	0.027	0.627	0.199	0.799	0.079	0.089
		CDA	0.702	0.516	0.057	0.611	0.484	0.602	0.273	0.671
	VPI	ASR	1.000	0.309	0.053	0.859	0.239	0.633	0.595	0.200
		CDA	0.922	0.468	0.146	0.802	0.900	0.588	0.794	0.889
	BadNet	ASR	1.000	0.726	0.065	0.673	0.094	0.235	0.829	0.132
		CDA	0.847	0.825	0.192	0.841	0.830	0.389	0.800	0.869

用损失（CDA 73.3%）。对于多触发器攻击 CBA，ProtoPurify 始终将 ASR 降至 16.5% 以下且 CDA 损失小于 3%；在 MNLI 的 BadNet 攻击上，ProtoPurify 进一步将 ASR 降至 1.6%/5.6%，同时保持 CDA 83.1%/81.7%，超越所有基线方法。

4.3 生成任务主要实验结果

表 4.6 展示了 ProtoPurify 在 Chat-Backdoor 生成基准上针对 DTBA、AutoPoison 和 VPI 三种攻击的实验结果。

表 4.6 生成任务实验结果。

		指标	后门模型	WAN	F/T	NAD	BEE	CROW	LET	PROTOPURIFY
Llama3-8B	DTBA	ASR	0.595	0.695	0.100	0.490	0.105	0.035	0.255	0.085
		CDA	0.810	0.330	0.880	0.820	0.790	0.490	0.820	0.805
	AutoPoison	ASR	0.800	0.067	0.210	0.618	0.050	0.227	0.143	0.065
		CDA	0.999	0.572	0.908	0.905	0.950	0.806	0.921	0.955
	VPI	ASR	0.990	0.230	0.190	0.105	0.095	0.150	0.120	0.080
		CDA	0.975	0.275	0.795	0.890	0.925	0.820	0.870	0.945
Mistral-7B	DTBA	ASR	0.770	0.735	0.130	0.765	0.050	0.525	0.685	0.060
		CDA	0.550	0.290	0.960	0.590	0.830	0.720	0.590	0.875
	AutoPoison	ASR	0.827	0.775	0.083	0.814	0.110	0.266	0.062	0.080
		CDA	0.999	0.901	0.824	0.906	0.960	0.835	0.890	0.960
	VPI	ASR	1.000	0.560	0.092	0.902	0.280	0.255	0.325	0.090
		CDA	0.990	0.735	0.750	0.935	0.920	0.785	0.885	0.950

值得注意的是，生成任务的评估基于在分类任务上构建的原型，验证了 ProtoPurify 跨任务领域的迁移能力。对于 DTBA 攻击，ProtoPurify 将 ASR 从 59.5%/77.0% 降至 8.5%/6.0%（Llama3-8B/Mistral-7B），同时 CDA 保持在 80.5%/87.5%。在 AutoPoison 和 VPI 攻击上，ProtoPurify 同样取得最低 ASR 和最高 CDA 的组合。

4.4 消融实验

4.4.1 良性向量移除的重要性

表 4.7 比较了使用 v_b （直接使用后门模型任务向量）与 $v_b - v_c$ （本文方法，减去良性任务向量）作为后门向量的效果差异。

结果清楚地表明了移除良性向量的重要性。直接使用 v_b 时，ASR 无法被有效降低，且 CDA 在多数设置下大幅下降，在 Emotion 数据集上甚至出现模型崩溃（ASR 和 CDA 均接近 0）。这支持了本文的假设： v_b 本质上混合了恶意的后门

表 4.7 后门向量构建策略对净化性能的影响（Llama3-8B）。

方法	指标	Emotion				SST-2			
		CBA	BadEdit	VPI	BadNet	CBA	BadEdit	VPI	BadNet
v_b	ASR	0	0.076	0	0.231	0.521	0.459	0.599	0.735
	CDA	0.060	0.244	0	0.155	0.834	0.818	0.806	0.790
$v_b - v_c$ （本文）	ASR	0.063	0.101	0.117	0.108	0.044	0.093	0.163	0.093
	CDA	0.902	0.492	0.461	0.868	0.891	0.859	0.880	0.905

相关更新和良性的任务相关更新，导致原型同时抑制恶意和良性信号，造成严重的效用损失。

4.4.2 聚合方法的影响

表 4.8 比较了算术均值（AM）和主成分分析（PCA）两种聚合方法对净化性能的影响。

表 4.8 聚合方法对后门净化性能的影响。

方法	指标	Emotion					SST-2				
		CBA	BadEdit	VPI	BadNet	均值	CBA	BadEdit	VPI	BadNet	均值
AM（本文）	ASR	0.063	0.101	0.117	0.108	0.097	0.044	0.093	0.163	0.093	0.098
	CDA	0.902	0.492	0.861	0.868	0.781	0.891	0.859	0.880	0.905	0.884
PCA	ASR	0.186	0.054	0.172	0.269	0.170	0.159	0.081	0.152	0.294	0.172
	CDA	0.827	0.418	0.813	0.796	0.714	0.725	0.807	0.761	0.820	0.778

总体而言，较简单的 AM 聚合始终优于 PCA。AM 方法将 ASR 平均降至 9.7%/9.8%，同时 CDA 保持在 78.1%/88.4%；而 PCA 方法的 ASR 较高（17.0%/17.2%），CDA 也有更大幅度的下降。这一性能差距可能源于 PCA 的目标不匹配：PCA 识别使后门向量方差最大化的方向，但这不一定与共享的后门相关信号对齐。此外，PCA 在均值中心化向量上操作，去除了对边界层检测和模型净化至关重要的一致均值偏移信号。因此，本文采用 AM 作为默认聚合函数。

4.4.3 边界层选择的影响

图 4.1 展示了不同边界层选择对净化性能的影响。

随着边界层向深层移动，受保护的底层范围增大，CDA 随之提升，而 1-ASR 下降（因参与净化的层减少）。本文基于显著性准则的边界层自动选择机制在 Emotion 数据集识别出第 4 层，在 SST-2 数据集识别出第 6 层，分别取得 ASR

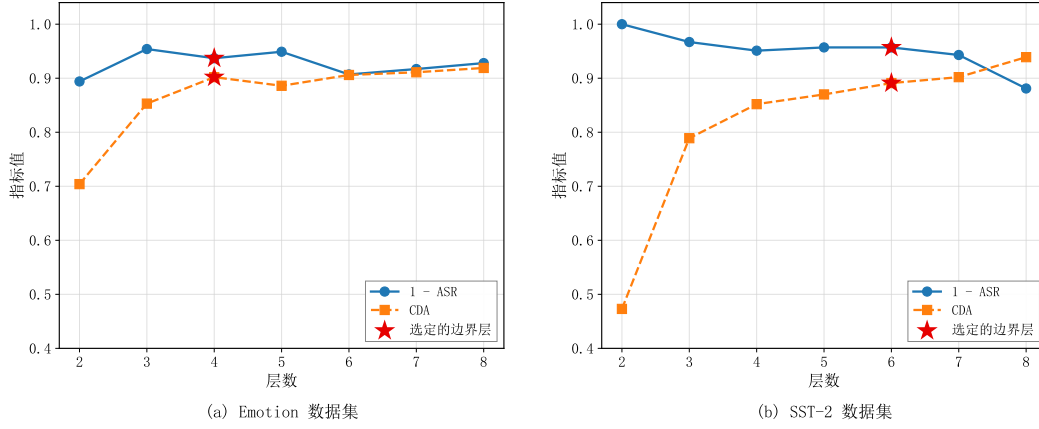


图 4.1 不同边界层选择对净化性能（1-ASR 和 CDA）的影响。星号标记本文方法自动选择的边界层。

6.3%/4.4% 和 CDA 90.2%/89.1% 的结果，在 ASR 降低与 CDA 保持之间实现了有效的权衡。

4.5 面向后门防御即服务的分析

4.5.1 可复用性

如表 4.4 和表 4.6 所示，ProtoPurify 展现了强大的可复用性。后门向量池通过离线模拟使用辅助数据集和已知攻击策略一次性构建，此后可被复用于有效净化所有使用相同架构的目标后门模型，即使这些模型在不同数据集、任务领域和后门攻击下训练。

4.5.2 可定制化

当防御方对后门攻击类型或下游任务有部分了解时，可以利用这些知识提升净化效果。在攻击感知设置中，针对性地使用与目标攻击对应触发器类型的 10 个后门向量构建原型，相比随机选取 10 个向量，可以稳定地取得更低 ASR 和更高 CDA。在任务感知设置中，纳入与目标任务相关的向量可分别为 Emotion 和 SST-2 数据集带来 8.0% 和 3.4% 的 ASR 下降，以及 4.0% 和 3.3% 的 CDA 提升。

4.5.3 可解释性

表 4.9 分析了 ProtoPurify 识别出的后门信号 c_i ，揭示了后门更新在模型内部的编码位置和方式。

表 4.9 干净模型与受损模型的后门信号分析（均值 $\pm$ 标准差）。

编号	粒度	干净模型	CBA (Δ_{CBA})	BadEdit (Δ_{BadEdit})
全局				
1	全部参数	0.491 ± 0.147	$0.540 \pm 0.102 (+0.049)$	$2.108 \pm 0.128 (+1.617)$
子层级				
2	自注意力	0.561 ± 0.187	$0.565 \pm 0.149 (+0.004)$	$2.151 \pm 0.151 (+1.590)$
3	MLP	0.398 ± 0.006	$0.505 \pm 0.030 (+0.107)$	$2.051 \pm 0.133 (+1.653)$
投影矩阵级				
4	q_proj	0.746 ± 0.017	$0.727 \pm 0.019 (-0.019)$	$2.204 \pm 0.166 (+1.458)$
5	k_proj	0.372 ± 0.014	$0.417 \pm 0.027 (+0.045)$	$2.260 \pm 0.125 (+1.888)$
6	v_proj	0.374 ± 0.009	$0.414 \pm 0.018 (+0.040)$	$2.105 \pm 0.112 (+1.731)$
7	o_proj	0.751 ± 0.012	$0.703 \pm 0.039 (-0.048)$	$2.034 \pm 0.135 (+1.283)$
8	up_proj	0.402 ± 0.007	$0.497 \pm 0.027 (+0.095)$	$2.133 \pm 0.126 (+1.731)$
9	$gate_proj$	0.397 ± 0.005	$0.508 \pm 0.023 (+0.111)$	$2.077 \pm 0.091 (+1.680)$
10	$down_proj$	0.396 ± 0.006	$0.511 \pm 0.041 (+0.115)$	$1.944 \pm 0.136 (+1.548)$

结果显示：（1）CBA 产生的信号变化较小（ $\Delta_{\text{CBA}} = +0.049$ ），而 BadEdit 产生的变化约大 2 个数量级（ $\Delta_{\text{BadEdit}} = +1.617$ ），这一统计特征有助于诊断攻击类型；（2）在子层级，信号增量主要集中在 MLP 块（ $+0.107$ ）而非自注意力（ $+0.004$ ），表明后门更新主要存储在 MLP 路径中（约 27 倍）；（3）在注意力投影级， k_proj 和 v_proj 呈现小幅正增量，而 q_proj 和 o_proj 呈现微弱甚至负增量，表明后门倾向于注入存储“内容”的投影矩阵（键和值），而非查询和输出矩阵；所有 MLP 投影（Up/Gate/Down）均呈现一致的增量。

4.5.4 运行时高效性

ProtoPurify 将净化过程分解为一次性离线阶段（阶段一）和每模型服务时阶段（阶段二至四）。服务时流程无需任何模型特定的梯度计算，每个模型的净化时间仅需 5–10 分钟。相比之下，F/T、NAD 和 LET 等基于训练的基线方法对每个目标模型进行迭代梯度更新，通常需要数百分钟的训练时间。

4.6 自适应攻击鲁棒性

本文考虑了知晓 ProtoPurify 防御策略的自适应攻击者，该攻击者通过预先放大后门信号 λ_i （Eq. (3.14)）来抵抗原型的净化效果。在原型未泄露设置和原型已泄露设置下，分别使用 1.2 倍放大因子（更强的放大会导致 CDA 严重下降

至不可用）进行测试。

如表 4.10 所示，在两种设置下，ProtoPurify 均将 ASR 保持在 10% 以下，且 CDA 与后门模型相当。这一鲁棒性源于攻击者面临的固有权衡：过强的放大会显著降低 CDA，使后门模型失去实用性；而较温和的放大则无法有效抵御 ProtoPurify 的净化。

表 4.10 ProtoPurify 对自适应后门攻击的鲁棒性（Llama3-8B）。

设置	指标	Emotion					SST-2				
		CBA	BadEdit	VPI	BadNet	均值	CBA	BadEdit	VPI	BadNet	均值
原始攻击	ASR	1.000	0.821	0.988	1.000	0.952	1.000	0.932	1.000	1.000	0.983
	CDA	0.938	0.534	0.949	0.887	0.827	0.957	0.951	0.964	0.954	0.957
原型未泄露 （自适应攻击）	ASR（攻击后）	1.000	0.764	0.263	0.912	0.735	1.000	0.850	0.772	0.853	0.869
	CDA（攻击后）	0.863	0.481	0.853	0.819	0.754	0.840	0.931	0.837	0.928	0.884
	ASR（净化后）	0.095	0.105	0.017	0.125	0.086	0.089	0.152	0.103	0.099	0.111
	CDA（净化后）	0.849	0.464	0.837	0.797	0.737	0.817	0.861	0.802	0.876	0.839
原型已泄露 （自适应攻击）	ASR（攻击后）	0.977	0.796	0.617	0.925	0.829	1.000	0.908	0.941	0.924	0.943
	CDA（攻击后）	0.955	0.518	0.872	0.835	0.795	0.956	0.896	0.948	0.929	0.932
	ASR（净化后）	0.107	0.100	0.088	0.016	0.078	0.078	0.142	0.069	0.031	0.080
	CDA（净化后）	0.910	0.506	0.838	0.812	0.767	0.885	0.829	0.921	0.917	0.888

4.7 干净模型稳定性

实用的 BDaaS 平台必须做到”安全优先”：当提交的模型是良性的（未经中毒训练），净化程序不应降低其干净任务效用。表 4.11 展示了将 ProtoPurify 应用于干净微调模型前后 CDA 的变化。

表 4.11 干净模型性能（CDA）分析。 ΔCDA 为应用 ProtoPurify 前后的绝对变化量。

数据集	干净模型 CDA	ProtoPurify 净化后 CDA	ΔCDA
Emotion	0.910	0.894	± 0.016
SST-2	0.928	0.902	± 0.026

ProtoPurify 在两个数据集上均仅引入可忽略不计的 CDA 变化。在无后门诱导的参数更新的情况下，干净模型向量与后门原型的投影很弱，导致净化干预极小，模型效用得以完整保留。

4.8 本章小结与全文结论

4.8.1 本章小结

本章在 Llama3-8B 和 Mistral-7B 两种大语言模型上，针对分类和生成两类任务，与 6 种代表性基线防御方法进行了全面的实验对比。实验结果表明，ProtoPurify 在以下方面均取得了优异表现：

1. **净化效果：**在所有数据集和攻击类型上，ProtoPurify 将 ASR 降低至 10% 以下，部分情况下低至 1.6%，同时 CDA 损失控制在 3% 以内，净化-效用综合表现超越所有基线方法。
2. **泛化能力：**尽管向量池仅由 BadNet 一种攻击构建，ProtoPurify 仍能有效抵御 5 种未见攻击；生成任务评估基于分类任务构建的原型，进一步验证了跨领域迁移能力。
3. **鲁棒性：**在原型未泄露和原型已泄露两种自适应攻击场景下，ASR 均保持在 10% 以下。
4. **安全性：**在干净模型上应用 ProtoPurify 仅引入极小的性能变化，满足 BDaaS“安全优先”的要求。

4.8.2 全文结论

本文提出了 ProtoPurify，一种面向后门防御即服务（BDaaS）的基于原型表示的大语言模型后门净化方法。ProtoPurify 从干净-后门模型对中提取后门原型，通过逐层原型对齐定位受影响的层，并通过以可控净化强度抑制原型对齐分量实现精细化净化。

本文的核心贡献在于：在无需了解触发器、训练数据或下游任务的极少先验假设条件下，实现了可复用、可定制、可解释且运行时高效的后门净化服务。在两种指令微调大语言模型上，针对分类和生成任务的广泛实验表明，ProtoPurify 在单触发器、多触发器、无触发器后门设置下，对 6 种代表性基线方法均取得了优越的净化-效用权衡。

局限性与未来工作：当前 ProtoPurify 的净化强度 α 是在整个模型层面统一施加的；未来工作可探索在不同子层（如 MLP 与注意力）或不同投影矩阵之间自适应调整净化强度，以进一步优化净化-效用权衡。此外，随着大语言模型架

构的持续演进，如何将 ProtoPurify 高效扩展至百亿以上参数量的模型也是值得深入探索的方向。

参考文献

- [1] BROWN T, MANN B, RYDER N, et al. Language models are few-shot learners [J]. Advances in Neural Information Processing Systems, 2020, 33: 1877-1901.
- [2] ACHIAM J, ADLER S, AGARWAL S, et al. GPT-4 technical report[A/OL]. arXiv (2023). <https://arxiv.org/abs/2303.08774>.
- [3] DUBEY A, JAUHRI A, PANDEY A, et al. The Llama 3 herd of models[A/OL]. arXiv (2024). <https://arxiv.org/abs/2407.21783>.
- [4] KURITA K, MICHEL P, NEUBIG G. Weight poisoning attacks on pre-trained models[C]//Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics. 2020: 2793-2806.
- [5] LI Y, LI T, CHEN K, et al. BadEdit: backdooring large language models by model editing[C]//International Conference on Learning Representations. 2024.
- [6] HUANG J, GENG L, WENG J. Composite backdoor attacks against large language models[C]//Findings of the Association for Computational Linguistics: ACL 2023. 2023.
- [7] HOU J, ZHAO Z, ZHANG R, et al. Dual-trigger backdoor attack on large language models[C]//Proceedings of the AAAI Conference on Artificial Intelligence. 2024.
- [8] YAN J, SEHWAG V, WANG C, et al. Virtual prompt injection for instruction-tuned large language models[C]//Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing. 2024.
- [9] HOU B, CHANG S. Distributed backdoor attacks on federated graph learning and defenses[C]//arXiv preprint arXiv:2405.14183. 2024.
- [10] LI Y, LYU X, KOREN N, et al. Neural attention distillation: erasing backdoor triggers from deep neural networks[C]//International Conference on Learning Representations. 2021.

- [11] YANG T, ZHANG N, ZHU H, et al. W2SDefense: weak-to-strong chain-of-thought reasoner defense against backdoor attacks[A/OL]. arXiv (2025). <https://arxiv.org/abs/2501.05671>.
- [12] LIU K, DOLAN-GAVITT B, GARG S. Fine-pruning: defending against backdooring attacks on deep neural networks[C]//International Symposium on Research in Attacks, Intrusions and Defenses. 2018: 273-294.
- [13] ZHU J, HONG L, LYU C, et al. PURE: unified purification for privacy, integrity and authenticity of cloud storage[C]//arXiv preprint arXiv:2405.06409. 2024.
- [14] LI B, WANG Y, WEI Z, et al. Obliviate: neutralizing task-agnostic backdoors within the parameter-efficient fine-tuning paradigm[C]//arXiv preprint arXiv:2409.14119. 2024.
- [15] MENG K, BAU D, ANDONIAN A, et al. Locating and editing factual associations in GPT[C]//Advances in Neural Information Processing Systems: Vol. 35. 2022: 17359-17372.
- [16] MENG K, SHARMA A S, ANDONIAN A, et al. Mass-editing memory in a transformer[C]//International Conference on Learning Representations. 2023.
- [17] ZENG Y, SUN W, NGUYEN T V D, et al. BEEAR: embedding-based adversarial removal of safety backdoors in instruction-tuned language models[C]//Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing. 2024.
- [18] MIN S, PARK H J, KIM I, et al. CROW: eliminating backdoors from large language models via internal consistency regularization[C]//arXiv preprint arXiv:2411.12768. 2024.
- [19] ILHARCO G, RIBEIRO M T, WORTSMAN M, et al. Editing models with task arithmetic[C]//International Conference on Learning Representations. 2023.
- [20] WORTSMAN M, ILHARCO G, GADRE S Y, et al. Model soups: averaging weights of multiple fine-tuned models improves accuracy and robustness[C]//

- Proceedings of the 39th International Conference on Machine Learning. 2022: 23965-23998.
- [21] JIANG A Q, SABLAYROLLES A, MENSCH A, et al. Mistral 7B[A/OL]. arXiv (2023). <https://arxiv.org/abs/2310.06825>.
- [22] VASWANI A, SHAZEER N, PARMAR N, et al. Attention is all you need[C]//Advances in Neural Information Processing Systems: Vol. 30. 2017.
- [23] HU E J, SHEN Y, WALLIS P, et al. LoRA: low-rank adaptation of large language models[C]//International Conference on Learning Representations. 2022.
- [24] OUYANG L, WU J, JIANG X, et al. Training language models to follow instructions with human feedback[C]//Advances in Neural Information Processing Systems: Vol. 35. 2022: 27730-27744.
- [25] QI F, LI M, CHEN Y, et al. Hidden killer: invisible textual backdoor attacks with syntactic trigger[C]//Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics. 2021: 443-453.
- [26] SUN M, LIU Z, BAIR A, et al. A simple and effective pruning approach for large language models[C]//International Conference on Learning Representations. 2024.

致谢

高嘉鑫

2026 年 5 月于珞珈山

